

EXPERIMENT - 26

1. Problem Statement

Write a C program that displays a menu and uses a switch statement to print the "Right-aligned pattern" selected by the user. The patterns have to be made with nested loops without using arrays, and each pattern may be written in a separate parameterless function with no return value.

- **Right-aligned increasing triangle**, input the number of rows n and print a right-aligned increasing triangle using asterisks.
- **Right-aligned increasing number triangle**, input n and print a right-aligned triangle of natural numbers, each row having the numbers from 1 up to that row number.
- **Right-aligned decreasing triangle**, input n and print a right-aligned decreasing triangle using asterisks.

2. Algorithm

Main Program

Step 1: START

Step 2: Display the menu: 1. Right-aligned increasing triangle, 2. Right-aligned increasing number triangle, 3. Right-aligned decreasing triangle.

Step 3: Read choice.

Step 4: Using a switch statement on choice:

 If choice is 1, call `rightAlignedIncreasing()`.

 If choice is 2, call `rightAlignedNumbers()`.

 If choice is 3, call `rightAlignedDecreasing()`.

Step 5: STOP

Function `rightAlignedIncreasing()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to $n - i$: print two blank spaces.

 Repeat for $j = 1$ to i : print an asterisk.

 Move to a new line.

Step 3: Return to the calling function.

Function `rightAlignedNumbers()`

Step 1: Read n .

Step 2: Repeat for $i = 1$ to n :

 Repeat for $j = 1$ to $n - i$: print two blank spaces.

 Repeat for $j = 1$ to i : print the value of j .

 Move to a new line.

Step 3: Return to the calling function.

Function rightAlignedDecreasing()

Step 1: Read n.

Step 2: Repeat for i = 1 to n:

Repeat for j = 1 to i - 1: print two blank spaces.

Repeat for j = 1 to n - i + 1: print an asterisk.

Move to a new line.

Step 3: Return to the calling function.

3. Test Case Table

#	Menu Choice	Input	Expected Output	Actual Output	Result
1	1. Right-aligned increasing	n = 4	1, 2, 3 and 4 asterisks, every row pushed to the right edge	Same	PASS
2	2. Right-aligned numbers	n = 4	1 / 1 2 / 1 2 3 / 1 2 3 4, all right aligned	Same	PASS
3	3. Right-aligned decreasing	n = 4	4, 3, 2 and 1 asterisks, the rows moving right as they shrink	Same	PASS
4	1. Right-aligned increasing	n = 1	A single asterisk with no blanks in front	Same	PASS
5	2. Right-aligned numbers	n = 6	Six rows, the last one being 1 2 3 4 5 6	Same	PASS

4. Source Code

rightaligned.c

```
/* Menu driven program that uses a switch statement to print the
   right-aligned pattern chosen by the user. Each pattern is written in
   its own function that takes nothing and returns nothing, and every
   pattern is made with nested loops. No arrays are used. */

#include <stdio.h>

void rightAlignedIncreasing(void);
void rightAlignedNumbers(void);
void rightAlignedDecreasing(void);

int main(void)
{
    int choice;

    printf("1. Right-aligned increasing triangle\n");
    printf("2. Right-aligned increasing number triangle\n");
    printf("3. Right-aligned decreasing triangle\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1: rightAlignedIncreasing(); break;
        case 2: rightAlignedNumbers(); break;
        case 3: rightAlignedDecreasing(); break;
    }

    return 0;
}

/* Row i has i asterisks, and the blanks in front push them to the
   right. Row i needs n - i blank places, and each blank is two spaces
   wide so that it lines up with the two characters of "** ". */
void rightAlignedIncreasing(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
}

/* Same shape as above, but instead of asterisks the row prints the
   natural numbers 1 up to the row number. */
void rightAlignedNumbers(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++)
            printf(" ");
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
}
```

```

/* Here the triangle shrinks, so the blanks grow instead. Row i needs
   i - 1 blank places and then n - i + 1 asterisks. */
void rightAlignedDecreasing(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

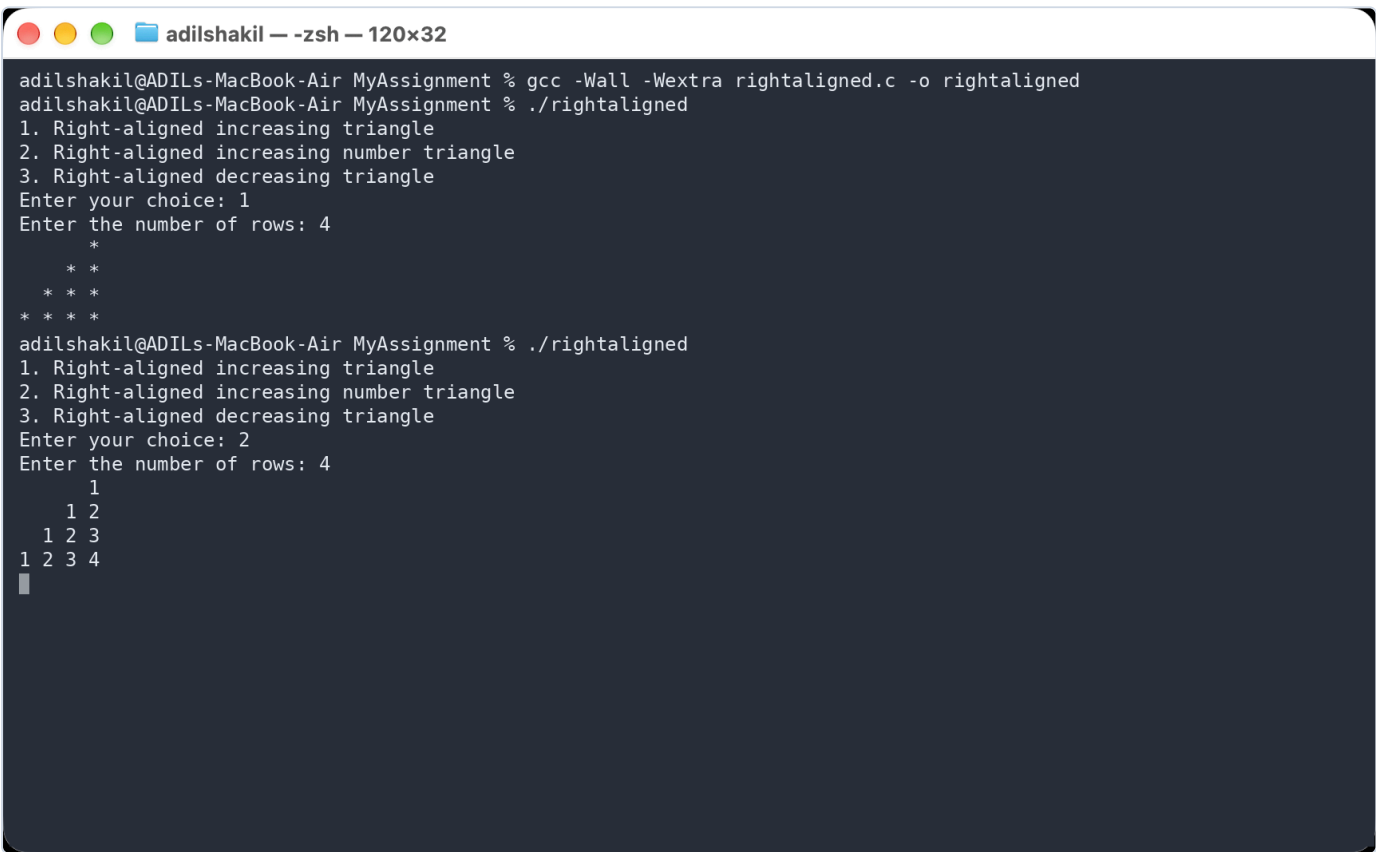
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= i - 1; j++)
            printf(" ");
        for (j = 1; j <= n - i + 1; j++)
            printf("* ");
        printf("\n");
    }
}

```

5. Compilation and Output

I compiled it with `gcc -Wall -Wextra rightaligned.c -o rightaligned` so that even the small warnings would show up, and it compiled clean in the first attempt itself. The screenshots below are of the runs I did on my own machine.

Patterns 1 and 2



```

adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra rightaligned.c -o rightaligned
adilshakil@ADILs-MacBook-Air MyAssignment % ./rightaligned
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 1
Enter the number of rows: 4
      *
     * *
    * * *
   * * * *
adilshakil@ADILs-MacBook-Air MyAssignment % ./rightaligned
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 2
Enter the number of rows: 4
      1
     1 2
    1 2 3
   1 2 3 4

```

Pattern 3, the Smallest Value of n, and a Bigger Triangle

```
adilshakil — zsh — 120x32
adilshakil@ADILs-MacBook-Air MyAssignment % ./rightaligned
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 3
Enter the number of rows: 4
* * *
 * * *
  * *
   *

adilshakil@ADILs-MacBook-Air MyAssignment % ./rightaligned
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 1
Enter the number of rows: 1
*

adilshakil@ADILs-MacBook-Air MyAssignment % ./rightaligned
1. Right-aligned increasing triangle
2. Right-aligned increasing number triangle
3. Right-aligned decreasing triangle
Enter your choice: 2
Enter the number of rows: 6
      1
     1 2
    1 2 3
   1 2 3 4
  1 2 3 4 5
 1 2 3 4 5 6
```

All five test cases came out right and the three patterns match the sample outputs given in the question exactly. The tricky part here was the alignment. My first idea was to print one space for every blank place, but then the triangle came out slanted, because a printed star actually takes two columns on the screen, the star and the space after it. Once I printed two spaces for every blank place instead of one, the right edge lined up properly.